30-DAY DAY-BY-DAY PLAN

Track 1 — Near-Term Interview Execution

Use this when interviews are near. Built for daily execution: 3–5 questions, two spoken answers, and one real-project connection — every day. Each day gives you what to **say** and the **trap** that separates a senior answer from a mid-level one.

· Goal & Rules

- ☐ Clear near-term coding and senior Java technical screens.
- ☐ **Coding method:** brute force → optimized → edge cases → complexity → dry run, out loud.
- ☐ **Theory method:** direct answer → mechanism → trade-offs → project example.
- ☐ **Output rule:** speak two answers out loud daily; get a real (human) mock in by the end of week 1.
- ☐ Do not skip project explanation — for lead roles it often decides the offer.

> **CONSCIOUS DSA CEILING**
>
> This plan targets the common contract bar — strings, arrays, HashMap, trees, searching, heap. If a target runs a harder algorithmic bar, add DP, graphs and intervals **deliberately** (see Track 4). Otherwise the time is better spent on Spring, Kafka and project depth.

Daily time split

AVAILABLE	HOW TO USE IT
60 min	35 min coding + 15 min Q&A spoken + 10 min project / weak-area log
90 min	50 min coding + 25 min Q&A and spoken answers + 15 min tracker
2+ hr	75 min coding / deep topic + 35 min Q&A and project + 20 min spoken answers + 10 min tracker

OPTIONAL Video or audio while commuting or on the treadmill — extra, never required.

· Week 1 — Coding Core

1 DAY

Strings / HashMap

Practice: first non-repeating char / duplicate characters / character frequency / anagram / string rotation

Say it (×2): Pick HashMap vs LinkedHashMap by whether order matters; state brute force first, then optimize.

> TRAP Override **equals() and hashCode() together** or HashMap lookups silently break; HashMap loses insertion order — use LinkedHashMap if order matters.

OPTIONAL Java string questions / immutability & string pool / StringBuilder vs StringBuffer

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

2 DAY — Arrays

Practice: second highest / move zeroes / two sum / remove duplicates / merge two sorted arrays

Say it (×2): Brute force first, then optimize with a HashMap or two pointers.

> TRAP Confirm whether the array is **sorted** before reaching for two pointers; watch **integer overflow** in sum-based logic.

OPTIONAL two-pointer technique / HashMap optimization / complexity basics

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

3 DAY — Sliding Window / Subarray

Practice: longest substring without repeat / Kadane / subarray with given sum / max sum subarray of size K / longest consecutive sequence

Say it (×2): Name the pattern before coding: fixed window, variable window, prefix sum, or Kadane.

> TRAP Decide **fixed vs variable window** up front — the bug is almost always in how you shrink the window.

OPTIONAL sliding window / prefix sum / Kadane

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

4 DAY — Linked List

Practice: find middle / reverse / detect cycle / merge two sorted lists / remove Nth from end

Say it (×2): Slow/fast pointer avoids needing length; the dummy node kills edge-case bugs.

> TRAP Use a **dummy node** for any op that can change the head; null-check before `.next`.

OPTIONAL slow/fast pointers / dummy node / Floyd cycle

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

5 DAY — LRU / Cache / HashMap Internals

Practice: LRU with LinkedHashMap / manual LRU (HashMap + DLL) / HashMap internals / equals/hashCode / cache with expiry

Say it (×2): Cache read-heavy data, never unsafe payment state.

> TRAP **LinkedHashMap(accessOrder=true)** gives LRU almost for free; a HashMap bucket treeifies at 8 entries (Java 8+).

DEEPER HashMap vs ConcurrentHashMap → Master Cheat Sheet

OPTIONAL LRU implementation / eviction strategies / hash collisions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

6 DAY — Threading

Practice: odd/even with two threads / producer-consumer (BlockingQueue) / deadlock + prevention / ExecutorService

Say it (×2): In production, prefer BlockingQueue and executors over manual wait/notify.

> TRAP **volatile = visibility, not atomicity** — use Atomic* or locks for compound actions.

OPTIONAL wait/notify / thread pools / BlockingQueue

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

7 DAY — Week 1 Mock MOCK

Practice: one string / one array / one linked list / one threading / explain complexity

Say it (×2): Speak while coding: approach, edge cases, complexity, dry run. **Do this with a human if at all possible** (Pramp / interviewing.io / a peer), and pull one random topic from a past day.

OPTIONAL coding mock / speaking while coding

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

· Week 2 — Streams, Search, Trees, Modern Java

8 DAY

Modern Streams — Employee Dataset JAVA 21

Practice: sort by salary / second-highest salary / group by department / max salary per dept / List to Map

Say it (×2): Streams help readability up to a point — don't force them on complex logic.

> TRAP This is modern Java, not just 8 — know `toList()`, `mapMulti`, `teeing`; streams are not automatically faster than a loop.

OPTIONAL groupingBy / map vs flatMap / Comparator.comparing

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

9 DAY

More Streams / Collections

Practice: word frequency / duplicate elements / sort map by value / joining / partitioning

Say it (×2): Know groupingBy, partitioningBy, the toMap merge function, and Optional.

> TRAP **toMap throws on duplicate keys** without a merge function; fail-fast iterators throw ConcurrentModificationException.

OPTIONAL Collectors / Optional / when streams hurt readability

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

10 DAY

Searching

Practice: binary search iterative/recursive / rotated sorted array / first/last occurrence / peak element

Say it (×2): Use overflow-safe mid = low + (high − low) / 2.

> TRAP Binary search is also a pattern on the **answer space** (min/max feasible value), not just on arrays.

OPTIONAL binary search pattern / edge cases

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

11 DAY

Trees / BST

Practice: inorder/preorder/postorder / level-order / validate BST / height / LCA

Say it (×2): BST inorder gives sorted order; validate with a min/max range, not just parent comparison.

> TRAP Comparing only parent vs immediate child **passes invalid trees** — carry a (min, max) range down the recursion.

OPTIONAL BFS vs DFS / recursion / min/max validation

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

12 DAY

Heap / PriorityQueue

Practice: kth largest / kth smallest / top-K frequent / merge K lists / median from stream (concept)

Say it (×2): For top-K, use a heap of size K: O(n log k).

> TRAP k-largest → **min-heap** of size k (not max-heap); k-smallest → max-heap. Easy to get backwards under pressure.

OPTIONAL PriorityQueue / top-K pattern / min vs max heap

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

13 DAY

Java 17 / 21

Practice: records / sealed classes / pattern matching / switch expressions / virtual threads vs platform threads / limitations of virtual threads

Say it (×2): Be honest — many enterprises still run 8/11/17. Virtual vs platform: cheap, JVM-scheduled, mounted on a small pool of carrier threads.

> TRAP **Virtual threads help IO-bound** concurrency, not CPU-bound; **don't pool** them. They **pin** the carrier inside `synchronized` or native calls — use locks instead. Records are shallow-immutable.

DEEPER Concurrency & JVM depth → Track 4

OPTIONAL what changed after Java 8 / virtual threads

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

14 DAY

Week 2 Mock MOCK

Practice: stream coding / binary search / tree / Java 17/21 explanation / one 25-min timed problem

Say it (×2): Solve, speak, test, stop rambling — the pressure is the point. Pull one random older topic too (spaced recall).

OPTIONAL senior coding mock

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

· Week 3 — Spring, JPA, Kafka

15 DAY

Spring Core

Practice: IoC/DI / constructor injection / bean lifecycle / scopes / stereotypes / @Bean vs @Component

Say it (×2): Constructor injection for required dependencies — testable and immutable.

> TRAP **@Bean** = explicit / third-party config; **@Component** = classpath-scanned. Field injection hides circular deps and breaks tests.

DEEPER SOLID / DIP → Track 2 Wk 1

OPTIONAL Spring core interview questions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

16 DAY

Spring Boot

Practice: auto-configuration / starters / profiles / external config / actuator / ControllerAdvice

Say it (×2): Boot = convention + auto-config + starters + embedded server + production tooling.

> TRAP Auto-config is **conditional** (@ConditionalOnClass / OnMissingBean...). Externalize config via profiles/env, never hardcode.

DEEPER 12-factor config → Track 2 Wk 2

OPTIONAL auto-configuration / exception handling

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

17 DAY

Transactions / JPA

Practice: @Transactional / propagation / isolation / self-invocation / N+1 / lazy/eager / locking

Say it (×2): Transactions are proxy-based — that is why self-invocation and private methods skip the proxy.

> TRAP Default propagation is **REQUIRED**; **self-invocation bypasses @Transactional**; fix N+1 with fetch join or @EntityGraph.

OPTIONAL transaction and JPA questions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

18 DAY

REST / Security

Practice: status codes / controller-service-repository / JWT / OAuth2 vs OIDC / filter chain / @Valid

Say it (×2): JWT validates claims/tokens; OIDC adds identity on top of OAuth2.

> TRAP Stateless auth = **no server session**; validate signature + expiry + audience on every request.

DEEPER Deeper auth (mTLS/PKCE) → Track 4 (Security)

OPTIONAL Spring Security JWT / REST design

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

19 DAY

Kafka Basics

Practice: producer/broker/topic/partition / consumer group / offset / commit / lag / partition key / how Kafka achieves high throughput

Say it (×2): Partitioning gives parallelism, and ordering within a partition. Throughput comes from sequential disk IO, zero-copy, batching and the OS page cache.

> TRAP **Ordering holds only within a partition**; the key picks the partition; more partitions = more parallelism but rebalancing cost.

OPTIONAL Kafka basics / consumer groups / how Kafka is fast

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

20 DAY

Kafka Senior

Practice: delivery semantics / duplicates / idempotent consumer / poison message / DLQ / rebalancing / broker failure (ISR + leader election) / Kafka vs RabbitMQ / safe event replay

Say it (×2): Real systems = at-least-once + idempotent processing. Broker dies → an in-sync replica is elected leader; with acks=all there is no data loss.

> TRAP Exactly-once is real **inside** Kafka (idempotent producer + transactions), but **end-to-end across external systems it is effectively at-least-once + dedup**. Say that distinction explicitly — it is the senior signal here.

DEEPER Resilience & saga/outbox → Day 23 / Track 2 Wk 6

OPTIONAL exactly-once / retries and DLQ / Kafka vs RabbitMQ

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

21 DAY

Week 3 Mock MOCK

Practice: Spring / JPA / Kafka / WSI project example / one coding problem

Say it (×2): Every framework answer carries one real project anchor. Pull one random older topic (spaced recall).

OPTIONAL Spring/Kafka mock

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

· Week 4 — Microservices, SQL, System Design, Projects

22 DAY

Microservices I

Practice: service decomposition / REST vs async / REST vs gRPC / API gateway / service discovery / config / load balancing

Say it (×2): Decide service boundaries by business capability and data ownership.

> TRAP **Each service owns its data** — no shared database. REST for sync, messaging for decoupled/async, gRPC for low-latency internal calls.

OPTIONAL microservices basics / API gateway / REST vs gRPC

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

23 DAY

Microservices II / Resilience

Practice: downstream failure / circuit breaker / retry / timeout / bulkhead / saga choreography vs orchestration / outbox / eventual consistency / drawbacks of microservices / debug a slow microservice

Say it (×2): Downstream failure = timeout + retry with backoff + circuit breaker + fallback, and idempotency so the retries are safe.

> TRAP **Retries without idempotency cause duplicates**; use a **saga** for distributed transactions, not 2PC. **Choreography** (events, decoupled) vs **orchestration** (a central coordinator, easier to reason about) — know the trade-off.

DEEPER Runbooks & observability → Track 2 Wk 6

OPTIONAL Resilience4j / saga choreography vs orchestration / outbox

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

24 DAY

SQL

Practice: second/Nth highest / max salary by dept / joins / group by / window functions / indexing / tuning

Say it (×2): Write queries by hand. Reach for window functions (ROW_NUMBER/RANK) for per-group ranking.

> TRAP **Index the WHERE / JOIN columns**; correlated subqueries are slow; HAVING filters aggregates, WHERE filters rows.

OPTIONAL SQL interview questions / window functions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

25 DAY

DB Scaling / NoSQL

Practice: RDBMS vs NoSQL / read replicas / sharding / partitioning / Cassandra vs PostgreSQL / CAP

Say it (×2): Relational scaling order: indexing → query tuning → read replicas → caching → partitioning → sharding.

> TRAP **Shard last**, not first; under a partition CAP forces a choice of consistency or availability; Cassandra is query-first (model to the read).

DEEPER Cassandra internals → Track 4 (Deep NoSQL)

OPTIONAL sharding / replication / CAP

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

26 DAY

System Design — eCommerce

Practice: shopping cart / checkout / gift-card inquiry / fraud/risk / idempotent payment API

Say it (×2): Closest to your WSI experience — own it. Sequence: requirements → capacity → API → data model → components → bottlenecks → trade-offs.

> TRAP Make **checkout and payment idempotent**; separate the read path (catalog/cart) from the write path (order/payment).

OPTIONAL design checkout / payment system

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

27 DAY

System Design — General

Practice: news feed / URL shortener / notification system / rate limiter / distributed cache

Say it (×2): Same sequence every time; state assumptions and rough QPS first, then find the bottleneck.

> TRAP Don't jump to the data model — **requirements and scale assumptions first**, or you design the wrong system.

OPTIONAL design Twitter / URL shortener

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

28 DAY

Project Deep Dive — WSI

Practice: cart/checkout / Accertify / Kafka / Tech Anchor / production issue / design decision

Say it (×2): Context → problem → your role → solution → impact (bounded number). Two minutes, no rambling.

> TRAP Own it in the **first person ("I")**; lead with a number; have the rejected alternative ready for the follow-up.

DEEPER STAR-plus template → Master §9

OPTIONAL senior project walkthrough

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

29 DAY

Project Deep Dive — CARB / Payments

Practice: Struts → Spring Boot / monolith → microservices / AWS migration / Cognito/OAuth/JWT / ISO8583

Say it (×2): Use CARB and payments as your older-depth stories.

> TRAP Keep the narrative to current and recent roles; in payments, idempotency rides on the **business / transaction key**.

OPTIONAL migration and payment questions

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

30 DAY

Full Mock MOCK

Practice: coding / Spring / Kafka / SQL / system design / project walkthrough / behavioral

Say it (×2): Clear opening line, structured body, real example, no rambling. Score yourself on the Master's Mock Scorecard. Pull two random older topics (spaced recall).

OPTIONAL full senior Java mock

Done: ☐ 3–5 questions ☐ spoke 2 answers ☐ linked to a project ☐ logged weak area

---

· Expanded Coding Practice Bank

Build the muscle to write these from scratch — plain loops, primitive arrays, no built-in collections. Each problem appears once, in its best home; work Tier 1 top-to-bottom first, then the topical sections (which reuse the same primitives). Do them without looking; if you stall, reach for a toolkit technique below, not a solution.

> **PLAIN-LOOP TOOLKIT (REACH FOR THESE, NOT A HASHMAP)**
>
> **Two pointers** from both ends — reverse, palindrome, in-place swap. · **Frequency array** — `int[256]` (ASCII) or `int[26]` (lowercase) replaces a HashMap for counting. · **One-pass running state** — track min / max / second-max / sum / count as you scan once. · **Write-index** — a second pointer marks where the next kept element goes (compaction in place). · **Prefix accumulation** — running sum or product. · **Fast / slow pointers** — middle of a list, cycle detection. · **Math tricks** — `n(n+1)/2` sum and **XOR** for missing / duplicate numbers. · **Brute force first** (nested loops), then optimize.

Tier 1 — Plain-Loop Fundamentals (no built-in data structures)

1. reverse (string / array / int)
2. palindrome (string / number)
3. character frequency (int[256])
4. first non-repeating char (two passes)
5. anagram check (int[26])
6. find duplicate characters
7. count vowels and consonants
8. remove duplicate characters (seen flags)
9. reverse words in a sentence (no split)
10. count words in a string
11. find max and min in one pass
12. find second largest in one pass
13. left-rotate an array by K (reversal)
14. remove duplicates from sorted array (in place)
15. move zeroes to end (in place)
16. sum and average without overflow
17. count occurrences of a value
18. naive substring search (strstr)
19. check two arrays are equal
20. missing number (sum or XOR)
21. single non-repeating number (XOR)
22. count digits and sum of digits
23. swap two numbers without temp
24. GCD (Euclid)
25. Fibonacci (two variables)
26. factorial (iterative)
27. prime check (trial division)
28. multiplication table

Strings

1. string rotation check
2. first repeated char
3. longest substring without repeat
4. longest substring with K distinct
5. run-length compression
6. run-length decode
7. longest common prefix
8. valid palindrome (ignore non-alphanumeric)
9. group anagrams
10. isomorphic strings
11. most repeated word
12. remove given characters
13. replace spaces with %20
14. capitalize each word
15. count substring occurrences
16. one-edit distance
17. longest palindromic substring (basic)
18. string to int (atoi)
19. int to string (itoa)
20. swap case

Numbers / Math

1. digital root
2. LCM
3. primes up to N (sieve)
4. Armstrong number
5. perfect number
6. count set bits
7. power(x, n) fast exponentiation
8. is power of two
9. decimal to binary
10. binary to decimal
11. reverse bits
12. add two number-strings
13. multiply without *
14. divide without /
15. sum of first N naturals
16. nth prime
17. max of two without if
18. trailing zeroes in factorial

Arrays

1. two sum
2. three sum (basic)
3. missing and repeating
4. merge two sorted arrays
5. sort 0/1/2 (Dutch flag)
6. leaders in array
7. majority element (Boyer-Moore)
8. max subarray (Kadane)
9. max product subarray
10. subarray with sum K
11. product except self
12. longest consecutive sequence
13. minimum absolute difference
14. equilibrium index
15. best time to buy/sell stock
16. trapping rain water
17. merge intervals
18. pair with given difference
19. rearrange positive and negative
20. next permutation
21. find the duplicate (Floyd)
22. count subarrays with sum K
23. min jumps to reach end

Matrix / 2D

1. transpose
2. rotate 90 degrees
3. spiral traversal
4. print boundary
5. search in sorted matrix
6. set matrix zeroes
7. diagonal sums
8. row with max ones
9. count islands (basic)
10. flood fill (basic)
11. matrix multiplication
12. symmetric matrix check
13. snake traversal
14. max sum submatrix (basic)
15. number of distinct paths

Searching & Sorting

1. linear search
2. binary search (iterative / recursive)
3. first and last occurrence
4. search in rotated sorted array
5. find peak element
6. integer square root (binary)
7. ceiling and floor in sorted
8. bubble sort
9. selection sort
10. insertion sort
11. merge sort
12. quick sort
13. counting sort
14. find rotation count (min in rotated)
15. allocate minimum pages (binary on answer)
16. min-max placement (binary on answer)
17. median of two sorted (basic)

Recursion / Backtracking

First re-solve factorial, Fibonacci, power and GCD from Tier 1 recursively — then:

1. tower of Hanoi
2. subsets / subsequences (power set)
3. permutations
4. combinations (n choose k)
5. generate parentheses
6. N-Queens (basic)
7. rat in a maze
8. word search in grid (basic)

Stack / Queue

1. implement stack with array
2. implement queue with array
3. min stack
4. valid parentheses / balanced brackets (multi-type)
5. next greater element
6. stack using two queues
7. queue using two stacks
8. evaluate postfix
9. infix to postfix (basic)
10. circular queue
11. sliding window maximum (deque)
12. largest rectangle in histogram
13. stock span

Linked List

1. reverse a linked list
2. find middle
3. detect cycle
4. find cycle start
5. merge two sorted
6. remove Nth from end
7. palindrome list
8. intersection node
9. delete node (given pointer)
10. reverse in K groups
11. add two numbers (lists)
12. remove duplicates (sorted)
13. remove duplicates (unsorted)
14. rotate by K
15. odd-even segregation
16. flatten (basic)

Trees / BST

1. inorder / preorder / postorder
2. level order (BFS)
3. height
4. diameter
5. count nodes and leaves
6. validate BST
7. lowest common ancestor
8. BST insert and search
9. BST delete (basic)
10. invert / mirror
11. left view and right view
12. zigzag traversal
13. root-to-leaf path sum
14. kth smallest in BST
15. balanced tree check
16. serialize / deserialize (basic)

Heap / PriorityQueue

1. kth largest / smallest
2. top-K frequent
3. merge K sorted lists

4. median from data stream
5. sort a K-sorted array
6. K closest points
7. connect ropes (min cost)
8. reorganize string (basic)
9. task scheduler (basic)

Concurrency

1. odd/even with two threads
2. print in sequence (3 threads)
3. producer-consumer (BlockingQueue)
4. producer-consumer (wait/notify)
5. create and prevent deadlock
6. thread-safe singleton (double-checked)
7. ExecutorService example
8. CompletableFuture composition
9. ConcurrentHashMap example
10. simple rate limiter
11. implement a bounded blocking queue
12. atomic counter vs synchronized

· Interview-Week Technical Q&A Drill

Speak each answer out loud with one project anchor. This is a drill list, not reading material.

Spring · Kafka · Microservices · SQL

1. Spring DI and constructor injection
2. Bean lifecycle and scopes
3. @Bean vs @Component
4. Spring Boot auto-configuration
5. @Transactional propagation / isolation
6. self-invocation transaction issue
7. N+1 and fetch strategies
8. JWT / OAuth / OIDC
9. Spring Security filter chain
10. REST status codes and error response
11. Kafka partitions and ordering
12. consumer groups and lag
13. offset commit strategies
14. delivery semantics
15. idempotent consumer
16. DLQ and poison messages
17. retry-storm prevention
18. how Kafka achieves high throughput
19. Kafka broker failure (ISR / leader election)
20. Kafka vs RabbitMQ
21. safe event replay
22. API gateway / service discovery
23. circuit breaker and fallback
24. saga and outbox
25. saga choreography vs orchestration
26. eventual consistency
27. REST vs gRPC
28. debug a slow microservice
29. idempotency-key design
30. SQL joins
31. window functions
32. indexing and query tuning
33. RDBMS vs NoSQL
34. scaling a relational DB
35. shopping-cart design
36. checkout design
37. rate-limiter design
38. notification-system design
39. SOLID / OCP
40. 12-factor app
41. observability / correlation ID
42. CI/CD rollback
43. a production-incident story

Collections & Internals

1. HashMap internals
2. HashMap vs ConcurrentHashMap
3. HashSet vs TreeSet
4. ArrayList vs LinkedList
5. LinkedHashMap vs TreeMap
6. CopyOnWriteArrayList
7. WeakHashMap use cases
8. fail-fast vs fail-safe iterators
9. why HashMap is not thread-safe
10. how ConcurrentHashMap is thread-safe
11. Comparable vs Comparator

Concurrency Q&A

1. CompletableFuture vs Future
2. synchronized vs ReentrantLock

3. AtomicInteger vs synchronized
4. CountDownLatch vs CyclicBarrier
5. what causes deadlocks
6. thread starvation vs livelock
7. CAS (compare-and-swap)
8. ExecutorService vs virtual threads
9. ThreadPoolExecutor tuning
10. volatile vs atomic

Modern Java (21 / 25)

1. virtual threads vs platform threads
2. when NOT to use virtual threads
3. structured concurrency
4. scoped values
5. records (what they solve)
6. pattern matching for switch
7. sealed types
8. sequenced collections
9. stream gatherers
10. string templates (preview)
11. foreign function & memory API

Core Java & OOP Fundamentals

1. pass-by-value (can you swap two ints?)
2. Integer caching (-128 to 127)
3. == vs equals
4. final vs finally vs finalize
5. immutability + writing an immutable class
6. why String is immutable / a good key
7. static method hiding vs overriding
8. overloading vs overriding
9. checked vs unchecked exceptions
10. Errors vs Exceptions (name 3 each)
11. constructor chaining (this / super)
12. marker interface (Serializable, Cloneable)
13. transient + Serialization
14. type erasure + primitives as generics
15. why no multiple inheritance (diamond)
16. BigDecimal for money (not double)
17. autoboxing / unboxing pitfalls
18. JDK vs JRE vs JVM
19. JVM memory: heap / stack / method area
20. String vs StringBuilder vs StringBuffer
21. Runnable vs Callable
22. tail recursion

· Track 1 Coverage Check

- ☐ Solved 20+ coding questions without looking at solutions.
- ☐ Can implement LRU or explain two approaches.
- ☐ Can handle the streams employee questions.
- ☐ Can answer Spring transaction and JPA N+1 questions.
- ☐ Can explain Kafka delivery semantics and duplicate handling.
- ☐ Can write SQL window-function queries.
- ☐ Can design cart/checkout/rate limiter at high level.
- ☐ Can explain WSI and CARB in under 2 minutes each.

> **COVERAGE IS NECESSARY, NOT SUFFICIENT**
>
> The real gauge is your Mock Scorecard trend in the Master. You are ready when you hit 4–5 consistently on questions you have not pre-seen — not when this list is full.